

OPTIMIZATION-I
PROJECT 2 - INTEGER PROGRAMMING

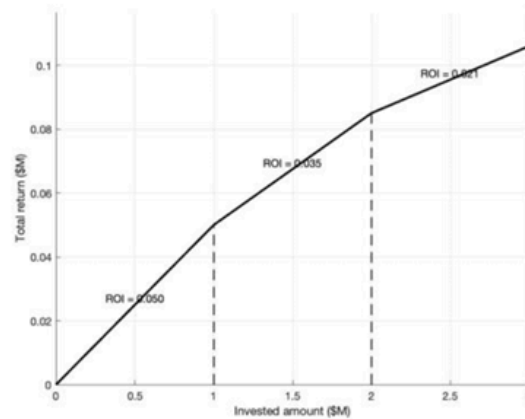
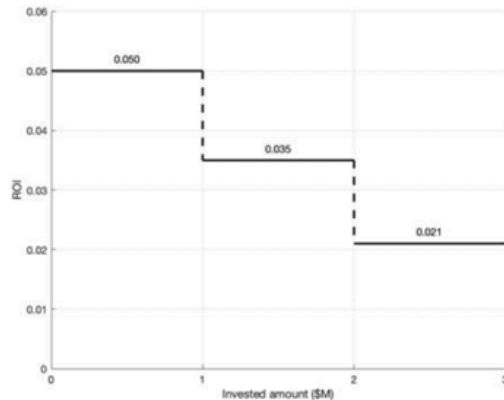
Optimizing Marketing Budgets



EXECUTIVE SUMMARY:

- This report presents a deep analysis of optimal marketing budget allocation across 10 advertising platforms.
- We analyzed two consulting firms' ROI estimates, evaluated allocation strategies under various constraints, implemented dynamic monthly optimization with reinvestment, and assessed budget stability.
- Best allocation delivers 5.4% returns- Invest heavily in Email, Instagram, and TV (each \$3M) plus AdWords (\$1M) to generate \$544K profit from the \$10M budget.
- Reinvesting returns creates huge gains- By reinvesting half of each month's profits, the budget grows to \$13.23M by December and total annual returns jump to \$7.08M
- Spending limits protect against mistakes- The \$3M cap per platform prevents putting all money in one channel and reduces losses if ROI estimates are wrong.

Part 1: Problem Context and Setup



The marketing department has been tasked to determine how to best allocate a \$10 million marketing budget across several marketing channels to maximize overall ROI. Further, the data provided by the consulting firms contain ROI estimates for each platform at different investment tiers. Each row represents a tier with a lower and upper investment bound and a constant ROI within that range. This tiered structure reflects diminishing returns—initial investments yield higher ROI, with subsequent investments in the same channel producing progressively lower returns.

Our objective is to determine the optimal budget allocation that maximizes total marketing return across all available channels.

Setup and Data Preparation: We first load the ROI data from the *roi_company1.csv*, and convert the upper bounds, lower bounds, and ROI values to appropriate numeric formats so that calculations work correctly. Each platform is limited to a maximum investment of \$3 million, which is enforced later as part of the model constraints.

Part 2: Concave ROI Model - Linear Program (Constraints)

The first firm's data show non-increasing ROI within each platform, meaning that additional spending gives diminishing returns. This pattern makes the total return concave in the amount invested, which allows us to model this problem as a linear program.

For each tier, we define a decision variable x_i (in millions) representing how much we invest in that tier. The objective is to maximize total return across all platforms: $\text{Max } \sum(\text{ROI}_i * x_i)$. In terms of constraints, the sum of all investments equals \$10 million, no platform may exceed \$3 million total, the amount spent on Print + TV $\leq$ Facebook + Email, and total social media spend (Facebook + LinkedIn + Instagram + Snapchat + Twitter) $\geq 2 \times$ (SEO + AdWords).

```
lp_mod.addConstr(x_lp.sum() == 10.0)

# per platform cap of 3M
for p in roi1['Platform'].unique():
    idx = [i for i, plat in enumerate(roi1['Platform']) if plat == p]
    lp_mod.addConstr(x_lp[idx].sum() <= 3.0)

# Print + TV ≤ Facebook + Email
print_idx = [i for i, p in enumerate(roi1['Platform']) if p in ['Print', 'TV']]
fb_email_idx = [i for i, p in enumerate(roi1['Platform']) if p in ['Facebook', 'Email']]
lp_mod.addConstr(x_lp[print_idx].sum() <= x_lp[fb_email_idx].sum())

# social ≥ 2 × (SEO + AdWords)
social_platforms = ['Facebook', 'LinkedIn', 'Instagram', 'Snapchat', 'Twitter']
search_platforms = ['SEO', 'AdWords']
social_idx = [i for i, p in enumerate(roi1['Platform']) if p in social_platforms]
search_idx = [i for i, p in enumerate(roi1['Platform']) if p in search_platforms]
lp_mod.addConstr(x_lp[social_idx].sum() >= 2.0 * x_lp[search_idx].sum())
```

Part 3: Concave ROI Model - Linear Program

We formulated a linear optimization problem using Gurobi, where each platform's ROI varies across investment tiers (reflecting diminishing returns). The model maximizes total return by selecting optimal investment amounts per tier while respecting all budget and strategic constraints.

```
lp_mod.Params.OutputFlag = 0
lp_mod.optimize()
```

Output:

Optimal Allocation for ROI

Print	0.00
TV	3.00
SEO	0.00
AdWords	1.00
Facebook	0.00
LinkedIn	0.00
Instagram	3.00
Snapchat	0.00
Twitter	0.00
Email	3.00

Key Findings:

- **Optimal Allocation:** When running this linear program, the allocation was optimal. The optimal allocation concentrates spending in TV, Instagram, and Email, with a smaller amount (\$1 million) in AdWords. No money is allocated to lower-performing platforms such as Print, Facebook, or LinkedIn, since those tiers have smaller marginal returns and would reduce efficiency.

- **Total Expected Return:** \$543,640 on \$10M investment (5.44% average ROI)

- **Strategic Insight:** With concave ROIs, the linear program naturally prioritizes the platforms with the strongest early-tier returns until they reach the \$3 million cap.

- **Constraint Impact:** Social media requirement (Instagram) and Print/TV constraint (satisfied by Email + TV balance) shaped the allocation

All business and budget constraints are satisfied in this linear program.

Part 4: Non-concave ROI Model - Mixed Integer Program

After reviewing the initial allocation, management sought a second opinion on ROI estimates. The second consulting firm's ROI data (*roi_company2.csv*) revealed non-concave ROI patterns—meaning higher returns can occur in later investment tiers rather than declining monotonically. This violates the diminishing returns assumption of the linear program, requiring a mixed integer programming (MIP) formulation using Special Ordered Set Type 2 (SOS2) constraints, to prevent the model from illogically investing in high-ROI tiers while skipping lower-investment tiers with lower ROIs.

For each marketing platform (Print, TV, Facebook, LinkedIn, etc.), we first built breakpoints for investment and total return. $bp[p]$ refers to cumulative investment levels (in \$ millions), starting at 0 and capped at \$3 million per platform. $val[p]$ refers to cumulative total returns at each breakpoint, computed as the running sum of ROI $\times$ investment width. This converts each platform's piecewise ROI table into a series of $(x, F(x))$ points, which is exactly what an SOS2 model needs.

```

# bp / val
platforms = roi2['Platform'].unique()
bp = {} # breakpoints of spend (in $M), starting at 0
val = {} # cumulative return at each breakpoint (in $M)
CAP = 3.0 # per platform cap

for p in platforms:
    tiers = roi2[roi2['Platform'] == p].sort_values('LowerBound')
    b_list = [0.0]
    v_list = [0.0]
    cur_b = 0.0
    cur_v = 0.0
    for _, r in tiers.iterrows():
        lb = float(r['LowerBound'])
        ub = float(min(r['UpperBound'], CAP))
        if ub <= cur_b:
            continue
        seg_lb = max(cur_b, lb)
        seg_ub = ub
        if seg_ub > seg_lb:
            width = seg_ub - seg_lb
            cur_b = seg_ub
            cur_v += width * float(r['ROI'])
            b_list.append(cur_b)
            v_list.append(cur_v)
        if cur_b >= CAP:
            break
    bp[p] = b_list
    val[p] = v_list

```

We then created the mixed integer program with $\lambda[p,k] \geq 0$, which is the convex-combination weights for the breakpoints of platform p , $\text{Spend}[p]$, which is the total dollars allocated to platform p , and $\text{Return}[p]$, which is the total return from that platform. Each set of λ 's must sum to 1 and obey an SOS2 restriction so only two adjacent λ 's can be positive, which ensures that Spend and Return interpolate correctly between breakpoints. We then defined Spend and Return as convex combinations of the breakpoint coordinates.

```

# SOS2
m2 = gp.Model('NonConcave_SOS2')

lam = {} # convex-combination weights per platform
Spend = {} # platform spend (0..CAP)
Return = {} # platform return

for p in platforms:
    K = len(bp[p])
    lam[p] = m2.addVars(K, lb=0.0, name=f"lam[{p}]")
    m2.addConstr(gp.quicksum(lam[p][k] for k in range(K)) == 1, name=f"lam_sum[{p}]")
    m2.addSOS(gp.GRB.SOS_TYPE2, [lam[p][k] for k in range(K)], bp[p])

    Spend[p] = m2.addVar(lb=0.0, ub=CAP, name=f"Spend[{p}]")
    Return[p] = m2.addVar(lb=0.0, name=f"Return[{p}]")
    m2.addConstr(Spend[p] == gp.quicksum(bp[p][k] * lam[p][k] for k in range(K)), name=f"Spend_def[{p}]")
    m2.addConstr(Return[p] == gp.quicksum(val[p][k] * lam[p][k] for k in range(K)), name=f"Return_def[{p}]")

```

Finally, the global constraints mirror those from the previous linear program: $\sum \text{Spend} = \$10$ million, each $\text{Spend}[p] \leq \$3$ million, $\text{Print} + \text{TV} \leq \text{Facebook} + \text{Email}$, and $\text{Social} (\text{Facebook} + \text{LinkedIn} + \text{Instagram} + \text{Snapchat} + \text{Twitter}) \geq 2 \times (\text{SEO} + \text{AdWords})$. And the objective is to

maximize the total return while satisfying these constraints. The code chunks below outline the steps for this mixed integer program.

```
# constraints
BUDGET = 10.0
m2.addConstr(gp.quicksum(Spend[p] for p in platforms) == BUDGET, name="budget")

PT = ['Print', 'TV']
FE = ['Facebook', 'Email']
SOCIAL = ['Facebook', 'LinkedIn', 'Instagram', 'Snapchat', 'Twitter']
SEARCH = ['SEO', 'AdWords']

m2.addConstr(gp.quicksum(Spend[p] for p in PT) <= gp.quicksum(Spend[p] for p in FE), name="pt_le_fe")
m2.addConstr(gp.quicksum(Spend[p] for p in SOCIAL) >= 2.0 * gp.quicksum(Spend[p] for p in SEARCH), name="social_ge_2x_search")
```

Output:

Optimal Allocation for ROI2

Print	3.00
TV	0.00
SEO	0.00
AdWords	2.33
Facebook	3.00
LinkedIn	1.67
Instagram	0.00
Snapchat	0.00
Twitter	0.00
Email	0.00

Key Findings:

- **Optimal Allocation and Strategic Shift:** Under the Company 2's non-concave ROI assumptions, we see that the platform allocation for the marketing budget shifts more heavily toward traditional media (Print) and paid search (AdWords), with 0 investment in Email and Instagram.

- **Total Expected Return:** \$452,800 on \$10M investment (4.53% average ROI). This is a lower overall return compared to the linear program from before (0.4528 vs 0.5436) and reflects the less favorable ROI structure in the new dataset.

- **ROI difference impact:** Non-concave ROI patterns led to a 17% lower return, highlighting the importance of accurate ROI estimation

In conclusion, the SOS2 formulation captures the full piecewise relationship between spending and return as each platform's total return now accounts for every tier up to the chosen investment

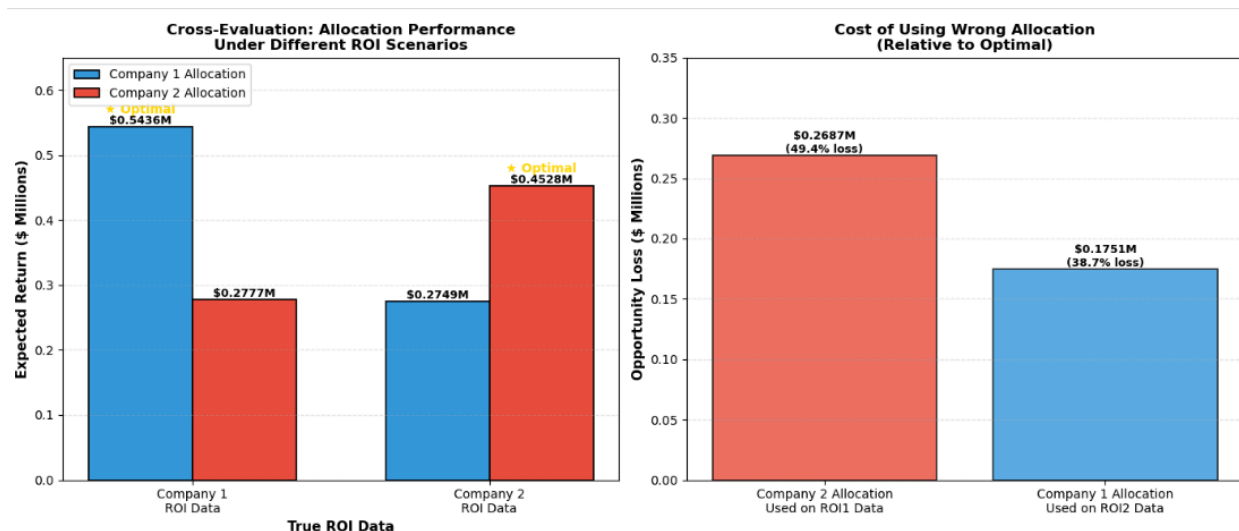
level rather than only its local ROI.

Overall, this model allocates the full \$10 million budget, satisfies all constraints, and provides a realistic and optimal recommendation for how to distribute marketing spend when ROI behavior is irregular or non-concave.

Part 5:

Given the conflicting ROI estimates from two consulting firms, we need to assess the robustness of each allocation strategy. Specifically, we evaluate how much value would be lost if we implement the wrong allocation—using Company 1's optimal strategy when Company 2's estimates are correct, or vice versa. This cross-validation reveals the risk exposure from estimation uncertainty and helps evaluate whether the \$3M per-platform constraint provides meaningful risk mitigation.

We perform a cross-evaluation by computing the expected return of each allocation strategy under both ROI scenarios. For each platform in a given allocation, we calculate returns tier-by-tier using the alternative firm's ROI data, tracking how investment fills each tier sequentially. The performance gap between the optimal objective and the cross-evaluated return quantifies the cost of model misspecification.



Key Findings:

- **Allocations differ substantially:**



- The two allocations are different and each performs substantially worse when evaluated under the other firm's ROI estimates. If ROI1 is correct but we use Company 2's allocation: Return drops 49.43% from \$543.6K to \$274.9K (loss of \$268.7K). If ROI2 is correct but we use Company 1's allocation: Return drops 38.67% from \$452.8K to \$277.7K (loss of \$175.1K)

- **Asymmetric risk:** Using the wrong allocation causes 39-49% value loss, demonstrating high sensitivity to ROI estimation accuracy.
- **Platform cap effectiveness:** The per-platform cap (the 3M max — our third constraint) is a useful risk-control measure: it prevents extreme concentration in a single channel and limits exposure to estimation error. Due to this constraint, for both allocations, the investment spread is across 3-4 channels rather than going all-in on a single platform.

In conclusion, the cross-evaluation reveals severe vulnerability to estimation error, with either misallocation costing \$175K-\$269K in forgone returns. The \$3M per-platform constraint proves valuable as a risk management tool—it forces diversification across channels, which partially hedges against ROI misestimation. Without this constraint, an optimizer might concentrate 70%+ of the budget into a single high-ROI channel, amplifying losses if that estimate is wrong. Thus, this constraint is indeed useful, as it bounds downside risk while allowing substantial investment in top-performing channels.

Part 6:

Marketing research reveals that each channel requires a minimum investment threshold to generate meaningful impact—investments below this threshold yield negligible returns. This introduces a binary 'invest or don't invest' decision for each platform: either allocate at least the minimum required amount or allocate nothing at all. We must reformulate the MIP to incorporate these minimum investment constraints while maintaining all previous strategic requirements.

Each platform now has a binary variable z_p :

- $z_p = 1$: You invest in that platform
- $z_p = 0$: You invest nothing
- If $z_p = 1$, total investment in that platform $\geq$ given minimum (from the CSV)

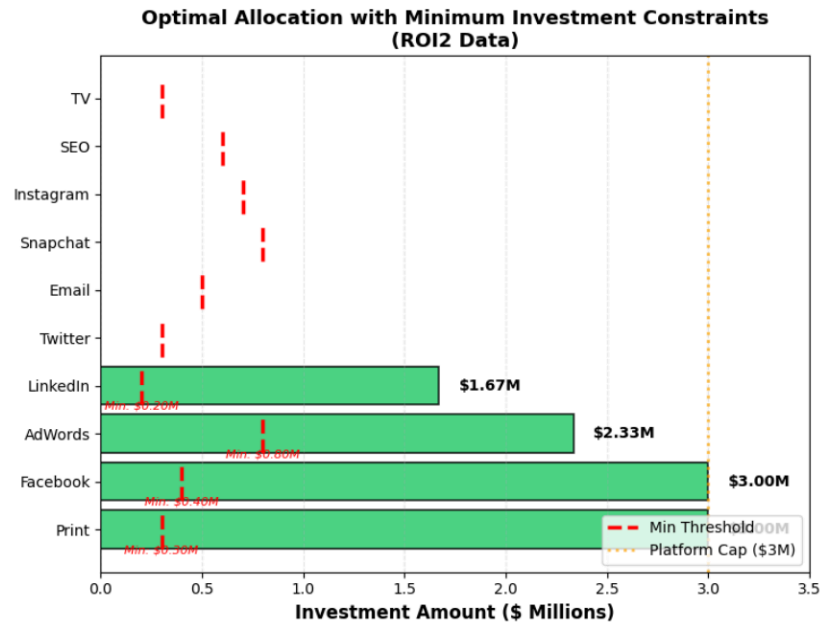
All previous constraints (budget, platform cap, social/print rules etc.) are kept.

```
#Link x and y within tiers
for i in range(len(roi2)):
    mip_min.addConstr(x_mip[i] <= widths2[i] * y_mip[i])

# Each platform chooses exactly one tier if invested, else none
for p in roi2['Platform'].unique():
    idx = [i for i, plat in enumerate(roi2['Platform']) if plat == p]
    mip_min.addConstr(gp.quicksum(y_mip[i] for i in idx) == z_p[p])
    # Cap per platform ≤ 3M
    mip_min.addConstr(gp.quicksum(roi2.loc[i, 'LowerBound'] * y_mip[i] + x_mip[i]
                                for i in idx) <= 3.0)
    # Minimum investment rule: if z_p=1, invest ≥ MinInvestment[p]
    mip_min.addConstr(gp.quicksum(roi2.loc[i, 'LowerBound'] * y_mip[i] + x_mip[i]
                                for i in idx) >= min_map[p] * z_p[p])
```

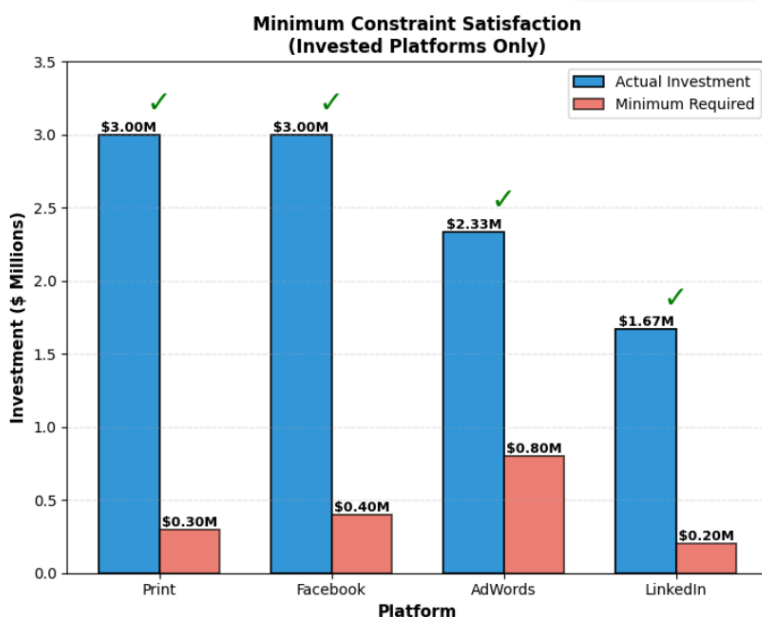

Output:

Print	3.00
TV	0.00
SEO	0.00
AdWords	2.33
Facebook	3.00
LinkedIn	1.67
Instagram	0.00
Snapchat	0.00
Twitter	0.00
Email	0.00



Key Findings:

- **Optimal allocation:** Print (\$3.00M), Facebook (\$3.00M), AdWords (\$2.33M), LinkedIn (\$1.67M)
- **Total expected return:** \$452,827 on \$10M investment (4.53% average ROI)
- **No change from Part 4:** The allocation is identical to the MIP solution, indicating the previous solution already satisfied all minimum investment thresholds.
- **Platforms invested:** 4 out of 10 platforms receive funding, with 3 hitting the \$3M cap



The minimum investment limits didn't change the optimal allocation, showing that the unconstrained model already invested meaningful amounts in each channel. This means the model naturally avoided small, ineffective allocations and preferred focusing the budget on fewer channels with larger investments. Any active minimum limits were effectively handled by the \$3M cap per platform, which encouraged diversification across channels.

Part 7:

The company has secured approval to reinvest 50% of monthly marketing returns into the following month's budget, creating a dynamic multi-period optimization problem. Starting with a \$10M budget in January, each month's performance compounds into subsequent periods—for example, a 5% January return increases February's budget to \$10.27M. We must determine optimal monthly allocations across all platforms while respecting all strategic constraints ($\text{Print} + \text{TV} \leq \text{Facebook} + \text{Email}$, $\text{Social} \geq 2 \times \text{Search}$, platform caps, and minimum investments) throughout the year.

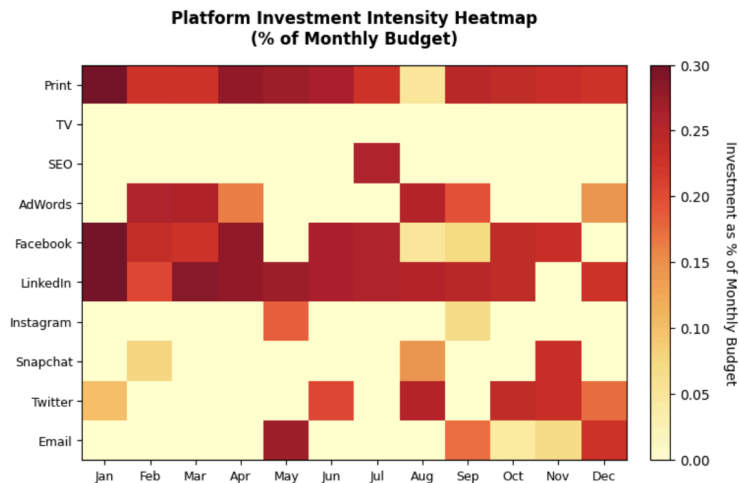
We solve this as a sequence of 12 independent MIP problems, one per month, where each month's budget depends on the previous month's realized return. For each month, we use the same MIP formulation from Part 6 (with platform-level binary variables z_p , tier-level variables x_i and y_i , and all strategic constraints), but with month-specific ROI data and dynamically updated budgets. The cumulative return base calculation accounts for prior tier returns within each platform, ensuring accurate objective function evaluation. After solving each month, we compute the total return, update the next month's budget as $\text{Budget_next} = \text{Budget_current} + 0.5 \times \text{Return_current}$, and proceed sequentially through the year.

Month	Budget	ROI(%)	Return(\$M)	Print	TV	SEO	AdWords	Facebook	LinkedIn	Instagram	Snapchat	Twitter	Email
Jan	10.0000	5.3942	0.5394	3.0000	0.0	0.0	0.0000	3.0000	3.0	0.0000	0.000	1.0000	0.0000
Feb	10.2697	4.0967	0.4207	2.3000	0.0	0.0	2.6566	2.4131	2.1	0.0000	0.800	0.0000	0.0000
Mar	10.4801	5.2363	0.5488	2.3920	0.0	0.0	2.6960	2.3920	3.0	0.0000	0.000	0.0000	0.0000
Apr	10.7545	4.8696	0.5237	3.0000	0.0	0.0	1.7545	3.0000	3.0	0.0000	0.000	0.0000	0.0000
May	11.0163	5.7328	0.6315	3.0000	0.0	0.0	0.0000	0.0000	3.0	2.0163	0.000	0.0000	3.0000
Jun	11.3321	4.6811	0.5305	3.0000	0.0	0.0	0.0000	3.0000	3.0	0.0000	0.000	2.3321	0.0000
Jul	11.5973	5.2717	0.6114	2.5973	0.0	3.0	0.0000	3.0000	3.0	0.0000	0.000	0.0000	0.0000
Aug	11.9030	5.1813	0.6167	0.6000	0.0	0.0	3.0000	0.6000	3.0	0.0000	1.703	3.0000	0.0000
Sep	12.2114	5.3383	0.6519	3.0000	0.0	0.0	2.3705	0.9000	3.0	0.8409	0.000	0.0000	2.1000
Oct	12.5373	5.9686	0.7483	3.0000	0.0	0.0	0.0000	3.0000	3.0	0.0000	0.000	3.0000	0.5373
Nov	12.9114	4.9878	0.6440	3.0000	0.0	0.0	0.0000	3.0000	0.0	0.0000	3.000	3.0000	0.9114
Dec	13.2334	4.6587	0.6165	3.0000	0.0	0.0	1.9334	0.0000	3.0	0.0000	0.000	2.3000	3.0000

Key Findings:

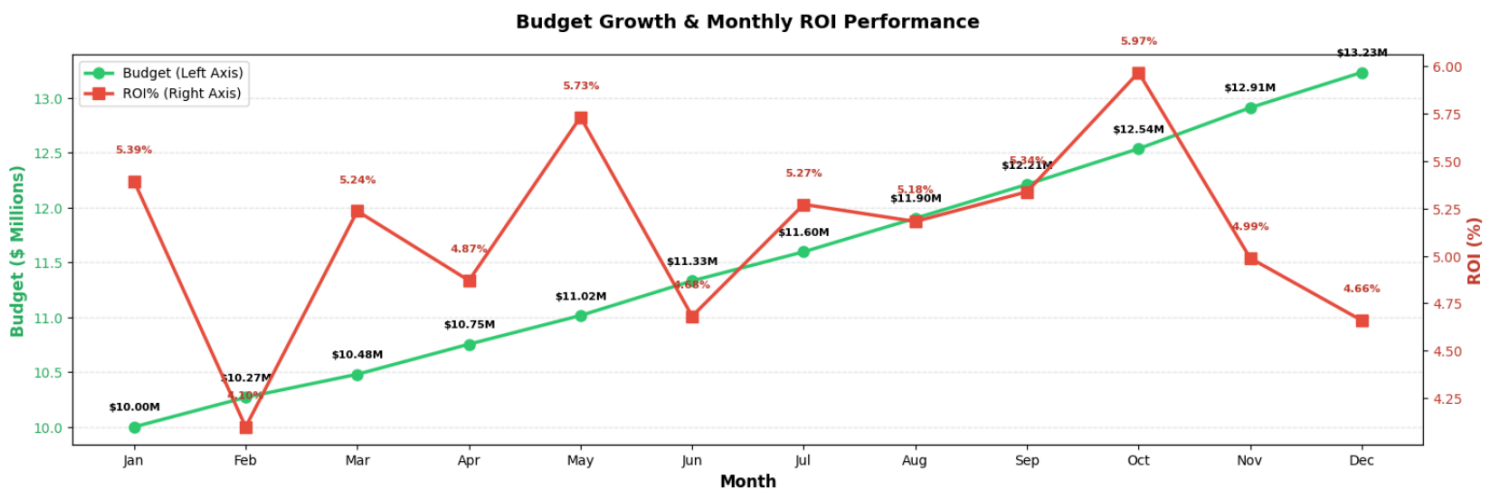
- **Budget growth:** Starting budget of \$10.0M grows to \$13.23M by December through consistent reinvestment of returns
- **Cumulative return:** Total return across 12 months is \$7.08M, representing a 5.12% average monthly ROI (70.8% of the initial \$10M budget)

- **Constraint binding:** The \$3M platform cap is frequently binding (Print hits cap in 8/12 months), while the $\text{Social} \geq 2 \times \text{Search}$ constraint shapes allocations significantly. Further, the minimum investment limits help avoid small, ineffective spends and ensure that each chosen platform receives a meaningful budget.



- Print is the most reliable channel, as it receives investment in all 12 months.
- Facebook receives investment in 10/12 months & LinkedIn invested in 11/12 months
- Allocations change monthly- Eg, AdWords receives \$0-\$3M depending on month, Instagram ranges from \$0-\$2.02M, showing responsiveness to ROI variations

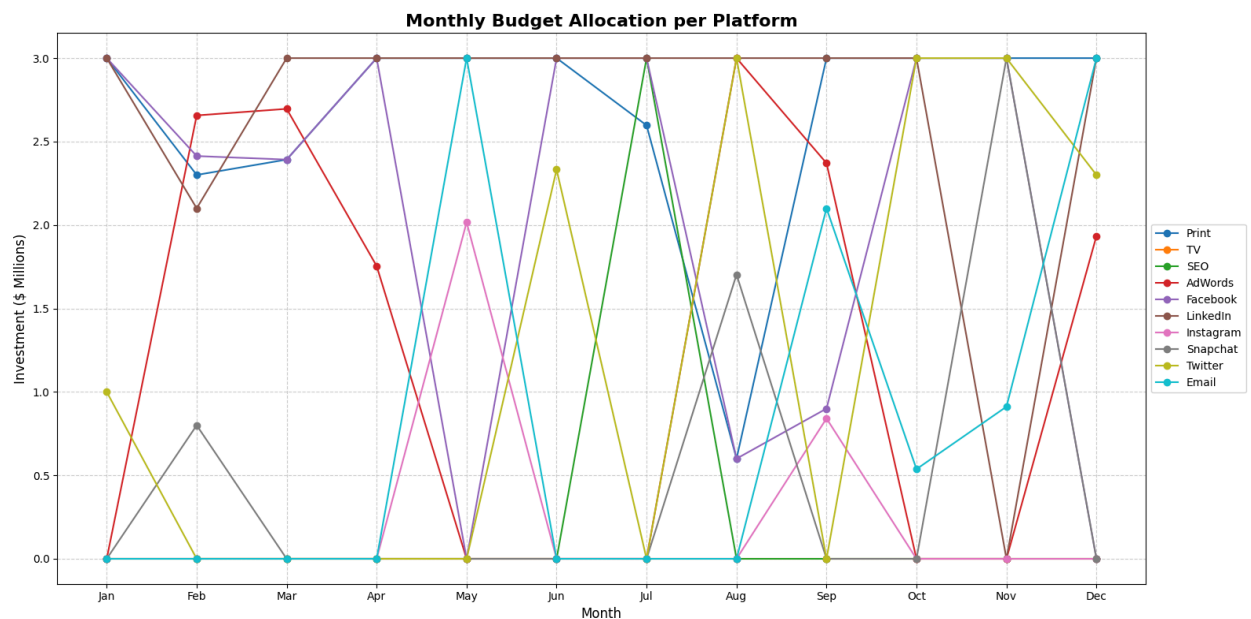
In conclusion, the reinvestment strategy shows strong compounding effects — the budget grows by 32.3% from January to December, greatly boosting returns. The model adjusts investments across 4–8 platforms each month based on which ones perform best, while still following all strategic rules..



Part 8:

In Part 7, the budget was optimized one month at a time. Each month was treated as a separate problem, with no consideration for previous or future months. Because of this, the model frequently shifted money from one platform to another whenever a slightly better ROI appeared.

Month-over-Month Change in Allocation (\$M)										
	Print	TV	SEO	AdWords	Facebook	LinkedIn	Instagram	Snapchat	Twitter	Email
Month										
Jan	nan	nan	nan	nan	nan	nan	nan	nan	nan	nan
Feb	0.70	0.00	0.00	2.66	0.59	0.90	0.00	0.80	1.00	0.00
Mar	0.09	0.00	0.00	0.04	0.02	0.90	0.00	0.80	0.00	0.00
Apr	0.61	0.00	0.00	0.94	0.61	0.00	0.00	0.00	0.00	0.00
May	0.00	0.00	0.00	1.75	3.00	0.00	2.02	0.00	0.00	3.00
Jun	0.00	0.00	0.00	0.00	3.00	0.00	2.02	0.00	2.33	3.00
Jul	0.40	0.00	3.00	0.00	0.00	0.00	0.00	0.00	2.33	0.00
Aug	2.00	0.00	3.00	3.00	2.40	0.00	0.00	1.70	3.00	0.00
Sep	2.40	0.00	0.00	0.63	0.30	0.00	0.84	1.70	3.00	2.10
Oct	0.00	0.00	0.00	2.37	2.10	0.00	0.84	0.00	3.00	1.56
Nov	0.00	0.00	0.00	0.00	0.00	3.00	0.00	3.00	0.00	0.37
Dec	0.00	0.00	0.00	1.93	3.00	3.00	0.00	3.00	0.70	2.09



This caused large and sudden jumps in spending. As observed from the table and image, 9 out of the 10 platforms (all except TV) changed by more than \$1M from one month to the next at least once. Several platforms even switched between \$3M and \$0 in a single month. These extreme swings show that the budget is highly unstable.

While this month-by-month approach does maximize returns for each individual month, it would be very difficult to implement in practice. Large monthly budget shifts create:

- **Operational challenges** (constantly signing and canceling advertising contracts)
- **Inconsistent marketing strategy** (no platform gets sustained investment)
- **Inefficient internal workflow** (teams must constantly redirect effort)

In short, the model behaves logically from a mathematical point of view, but the results are not realistic for real-world marketing planning. This happens because the model is myopic, meaning it optimizes one month at a time and does not consider long-term stability. Since there is no penalty for sudden changes, the model moves money quickly whenever the estimated ROI shifts.

Proposed Solution

To fix this, we suggest using a 12-month optimization model that decides all monthly budgets at once. In this model, we include stability constraints that limit how much each platform's budget can change month-to-month (no more than \$1M difference from one month to the next). Then, for every platform p and every month $t > 1$, we would add two new constraints:

$$x[p, t] - x[p, t-1] \leq 1.0$$

$$x[p, t] - x[p, t-1] \geq -1.0$$

This approach may slightly reduce total returns, but it creates a more stable, realistic, and strategically consistent budget that the marketing team can actually execute.